ACTA DE EVIDENCIA DE PRUEBAS DE FRONTEND (UI / UX)

Sistema: Cliente Web – Gestor de Proyectos (Ejercicio5Cliente)

1. Información General de la Evaluación

Proyecto / Aplicación	Cliente Web Gestor de Proyectos (Ejercicio5Cliente)
Tipo de Pruebas	Pruebas Funcionales de Frontend / Interfaz de Usuario (UI/UX)
Arquitectura / Stack	Spring Boot · Thymeleaf · Bootstrap 5 · jQuery · CSS3
Navegadores Evaluados	Google Chrome / Microsoft Edge / Mozilla Firefox
Resoluciones de Pantalla	Desktop (1920x1080) / Vista Adaptable (Tablet y Mobile)
Sprint / Iteración	Unidad 2 – Ejercicio 5
Responsable de Pruebas	Brandon Gustavo Mendoza Amaro
Fecha de Evaluación	Octubre 2026

2. Resumen Ejecutivo (Métricas de Calidad de Frontend)

Métrica de Calidad	Valor Obtenido
Total de Escenarios Evaluados	20
Escenarios Exitosos (Passed)	17
Escenarios con Observaciones / Errores	3
Porcentaje de Conformidad UI/UX	85%
Estado de la Integración con Backend (API)	Conectado (localhost:8081)
Resultado General de la Evaluación	✅ Casi Aprobado.

3. Matriz de Casos de Prueba de Frontend

Se ejecutaron los 20 escenarios de la interfaz del cliente web y se identificó, para cada uno, el endpoint del backend con el que se comunica; cuando el escenario no consume ningún servicio (vistas estáticas o de diseño) se indica "No tiene". En la sección 3.1 se resumen los cinco flujos base de la práctica y en la sección 3.2 se presenta la matriz completa.

3.1 Casos de integración (flujos base)

ID	Flujo	Acción	Endpoint	Resultado esperado	Caso UI / Estado
INT-01	Consulta	Cargar el dashboard de proyectos (/inicio)	POST /buscarProyectos	Los datos se reciben y se muestran correctamente en tarjetas	UI-09 ✅ Pasa
INT-02	Registro	Crear un proyecto desde el modal con datos válidos	POST /addProyecto	El registro se crea y la interfaz confirma la operación	UI-10 ✅ Pasa
INT-03	Actualización	Editar un proyecto desde el modal con datos precargados	POST /updateProyecto	La información se actualiza correctamente	UI-11 ✅ Pasa
INT-04	Eliminación	Eliminar un proyecto confirmando la advertencia	DELETE /deleteProyecto	El registro se elimina y la vista se actualiza	UI-12 ✅ Pasa
INT-05	Error controlado	Iniciar sesión con credenciales incorrectas o vacías	POST /getUsuarioLogin	La interfaz controla y muestra adecuadamente el error	UI-05 ✅ Pasa

3.2 Matriz completa de casos de prueba de frontend con su endpoint

ID	Vista / Módulo	Escenario de interfaz evaluado	Resultado visual esperado	Endpoint relacionado	Estado
UI-01	Portada / Bienvenida	Visualización de página principal (home.html), banners, logotipo y accesos directos	Página cargada sin errores visuales ni enlaces rotos	No tiene (vista estática)	✅ Pasa
UI-02	Manual de Usuario	Acceso a la vista de manual (/manual), reproducción de video o visualización de guía	Contenido multimedia o guía legible y controles funcionales	No tiene (vista estática)	✅ Pasa
UI-03	Tutorial de la Plataforma	Acceso a la sección de tutorial (/tutorial), carga de componentes interactivos	Instrucciones y recursos visuales desplegados correctamente	No tiene (vista estática)	✅ Pasa
UI-04	Inicio de Sesión (Login)	Despliegue del formulario de login (/login) con campos correo, contraseña y botón	Elementos alineados, estilos responsivos y campos interactivos	No tiene (solo despliega el formulario)	✅ Pasa
UI-05	Login: Validación de Errores	Intento de inicio de sesión con credenciales incorrectas o vacías	Notificación de alerta en pantalla indicando datos inválidos	POST /getUsuarioLogin	✅ Pasa
UI-06	Login: Autenticación Exitosa	Ingreso con credenciales válidas y redirección automática al dashboard (/inicio)	Autenticación correcta y redirección fluida a la vista de inicio	POST /getUsuarioLogin	✅ Pasa

ID	Vista / Módulo	Escenario de interfaz evaluado	Resultado visual esperado	Endpoint relacionado	Estado
UI-07	Registro de Usuario	Despliegue del formulario de registro (/register) con nombre, correo, rol y avatar/color	Formulario completo, selector de color interactivo y validaciones	No tiene (al desplegar; el envío usa POST /addUsuario, ver UI-08)	❌ Falla: Campos faltantes
UI-08	Confirmación de Registro	Envío exitoso del formulario de registro y redirección al login	Usuario registrado en base de datos y retorno exitoso al login	POST /addUsuario	✅ Pasa
UI-09	Dashboard de Proyectos	Visualización de la cuadrícula o tarjetas de proyectos registrados (/inicio)	Renderizado de tarjetas con nombre, descripción, estado y fondo	POST /buscarProyectos	✅ Pasa
UI-10	Crear Nuevo Proyecto	Apertura del modal para crear proyecto, llenado de datos y confirmación	El nuevo proyecto aparece en la lista sin necesidad de reinicio	POST /addProyecto	✅ Pasa
UI-11	Editar Proyecto	Apertura del modal de edición con los datos precargados del proyecto seleccionado	Datos actualizados reflejados de inmediato en la tarjeta	POST /updateProyecto	✅ Pasa
UI-12	Eliminar Proyecto	Activación del botón eliminar proyecto, confirmación de advertencia y remoción	Proyecto eliminado y la cuadrícula se actualiza automáticamente	DELETE /deleteProyecto	✅ Pasa
UI-13	Tablero del Proyecto	Acceso al tablero de proyecto (/proyecto?id=...) con columnas de estado de tareas	Cabecera del proyecto visible y columnas organizadas por estado	POST /getTarea	✅ Pasa

ID	Vista / Módulo	Escenario de interfaz evaluado	Resultado visual esperado	Endpoint relacionado	Estado
UI-14	Crear Nueva Tarea	Modal para crear tarea con título, descripción, fecha de vencimiento y usuario asignado	Tarea registrada exitosamente asociada al proyecto actual	POST /addTareas POST /addAsignacion (al asignar integrante)	✅ Pasa
UI-15	Distribución de Tareas	Visualización de tarjetas de tareas en sus respectivas columnas (Por hacer, En proceso, Terminado)	Colores e insignias de estado (success, warning, danger) correctos	POST /getTarea	✅ Pasa
UI-16	Actualizar Tarea	Edición de información de la tarea o cambio de estado / columna	Cambio guardado y reflejado visualmente en el tablero	POST /updateTareas POST /updateAsignacion (al reasignar integrante)	❌ Falla: Faltan campos
UI-17	Eliminar Tarea	Ejecución de la acción de eliminar tarea en el tablero	Tarjeta de tarea removida correctamente del proyecto	DELETE /deleteTareas	✅ Pasa
UI-18	Ver Comentarios de Tarea	Apertura del panel o modal de comentarios correspondiente a una tarea específica	Listado cronológico de comentarios previos del equipo	POST /getComentario	✅ Pasa
UI-19	Agregar Nuevo Comentario	Escritura y envío de un nuevo comentario en la tarea	Comentario añadido al historial con fecha y autor en tiempo real	POST /addComentario	✅ Pasa

ID	Vista / Módulo	Escenario de interfaz evaluado	Resultado visual esperado	Endpoint relacionado	Estado
UI-20	Diseño Adaptable (Responsive)	Comprobación de la interfaz en resolución móvil / tablet o ventana reducida	Menú hamburguesa colapsable, tarjetas adaptadas sin desborde	No tiene (diseño visual)	✅ Pasa

4. Evidencia Visual – Capturas de Pantalla de la Interfaz

4.1 Evidencia visual de los flujos ejecutados

Capturas de la interfaz de cada caso, identificadas con su ID. Se indica el endpoint con el que se relaciona cada flujo.

Módulo: Portada, Bienvenida y Soporte

[UI-01] Página de Inicio / Landing (home.html)

Objetivo: Captura de la vista principal con cabecera, banners descriptivos y botones de acción.

Endpoint relacionado: No tiene

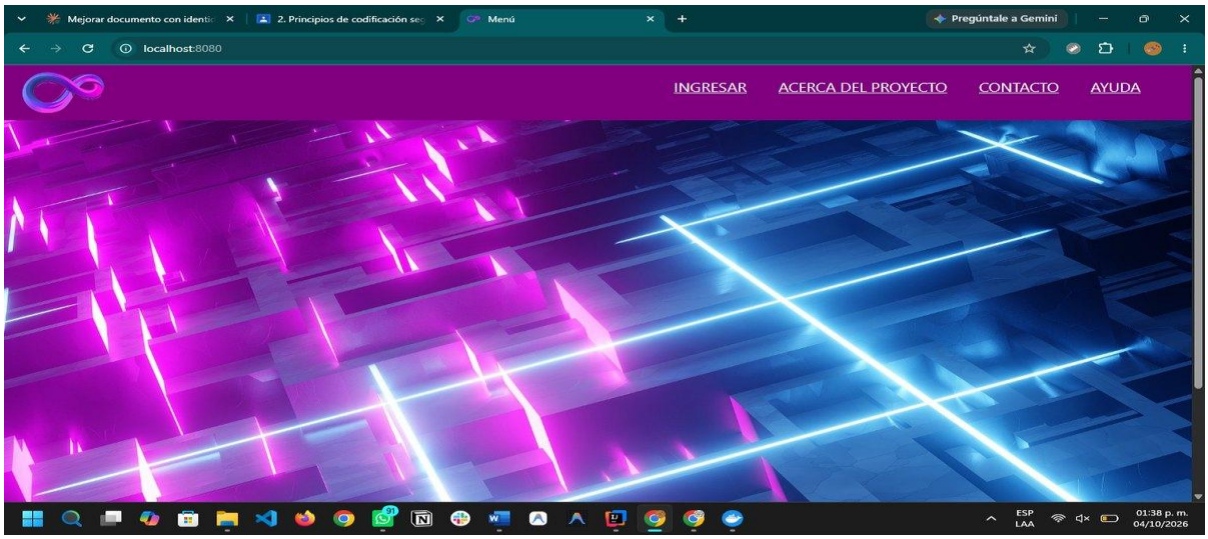


Figura UI-01.

[UI-02] Manual de Usuario (manual.html)

Objetivo: Captura de la vista del manual de usuario con guía de uso o reproductor multimedia.

Endpoint relacionado: No tiene

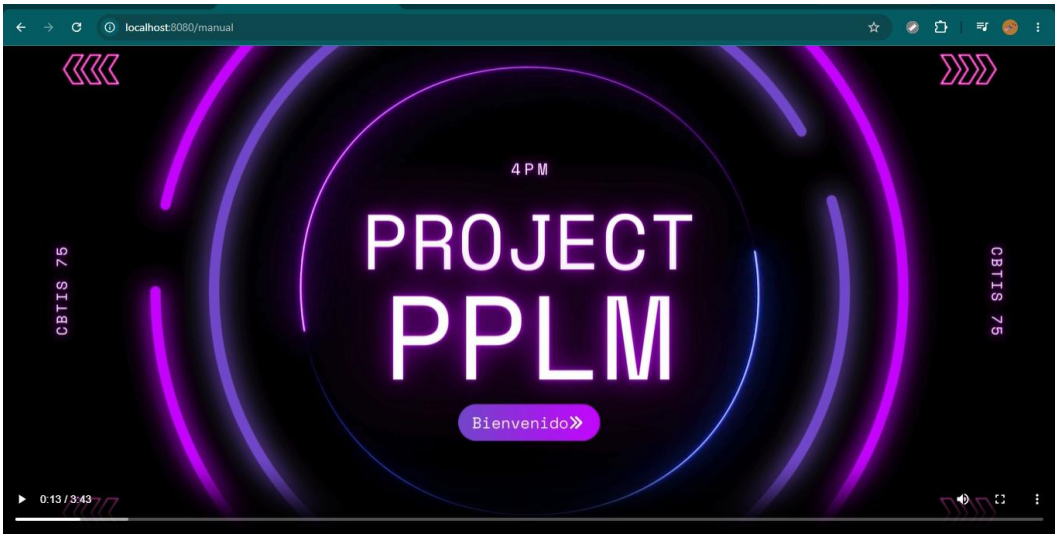


Figura UI-02.

[UI-03] Tutorial de la Plataforma (tutorial.html)

Objetivo: Captura de la vista de tutorial interactivo o video instructivo.

Endpoint relacionado: No tiene

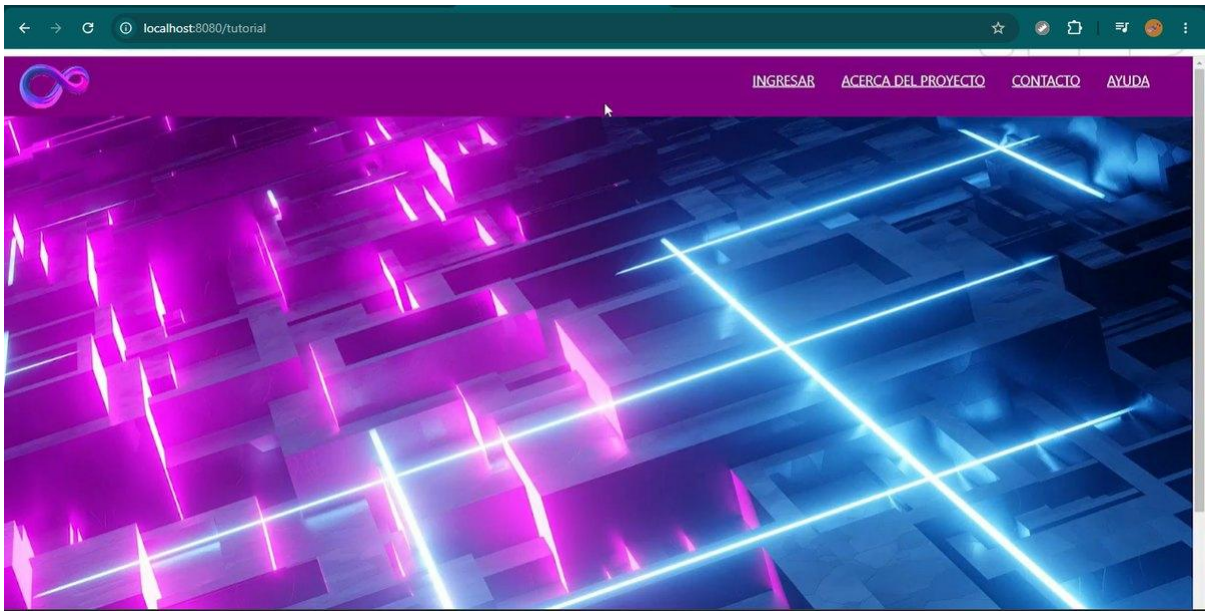


Figura UI-03.

Módulo: Autenticación y Cuentas de Usuario

[UI-04] Interfaz de Login (login.html)

Objetivo: Captura del formulario de inicio de sesión con campos de correo y contraseña.

Endpoint relacionado: No tiene (solo despliega el formulario)

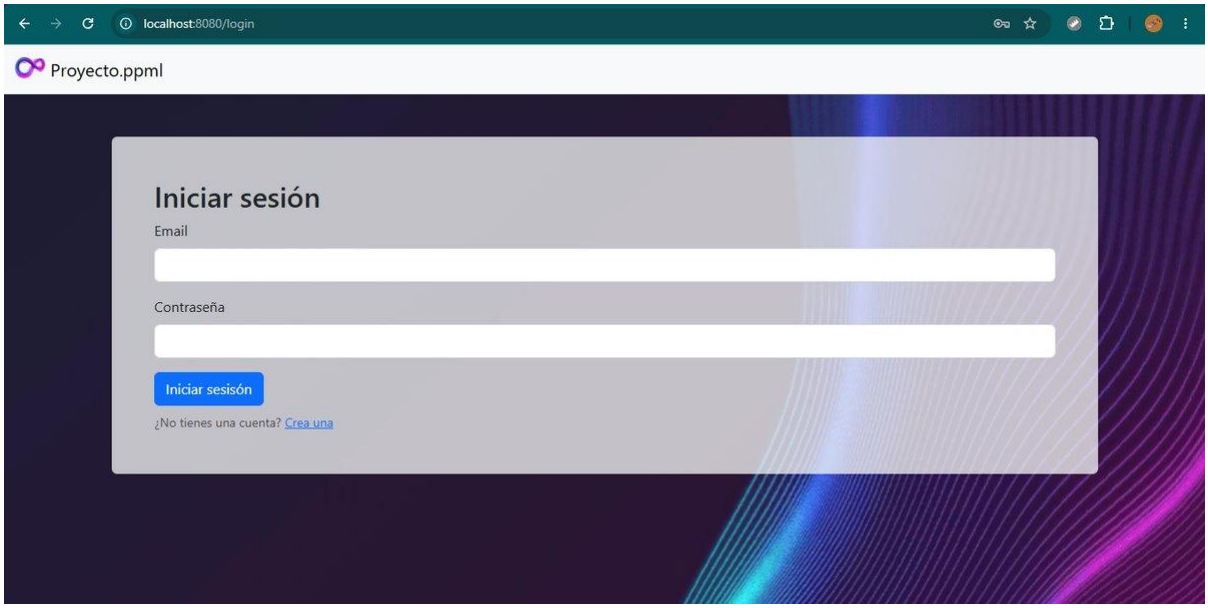


Figura UI-04.

[UI-05] Login: Alerta de Credenciales Inválidas

Objetivo: Captura de la alerta o mensaje de error en pantalla al ingresar datos incorrectos.

Endpoint relacionado: POST /getUsuarioLogin

Nota: Está mal escrito el nombre del botón para iniciar sesión.

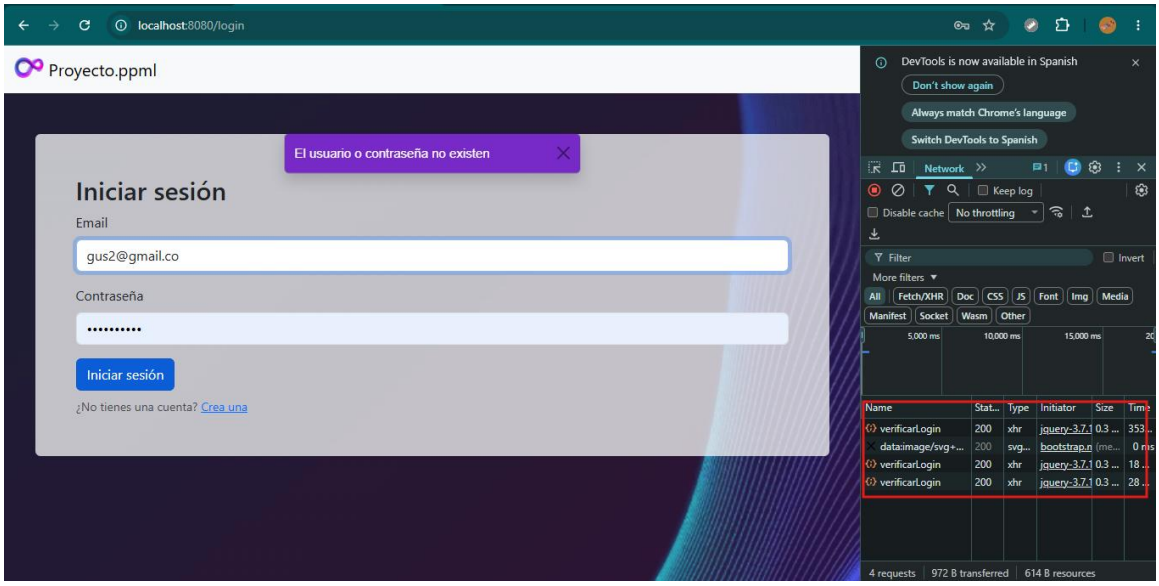


Figura UI-05.

[UI-06] Login Exitoso y Redirección al Dashboard

Objetivo: Captura de la transición o ingreso satisfactorio al panel de usuario autenticado.

Endpoint relacionado: POST /getUsuarioLogin

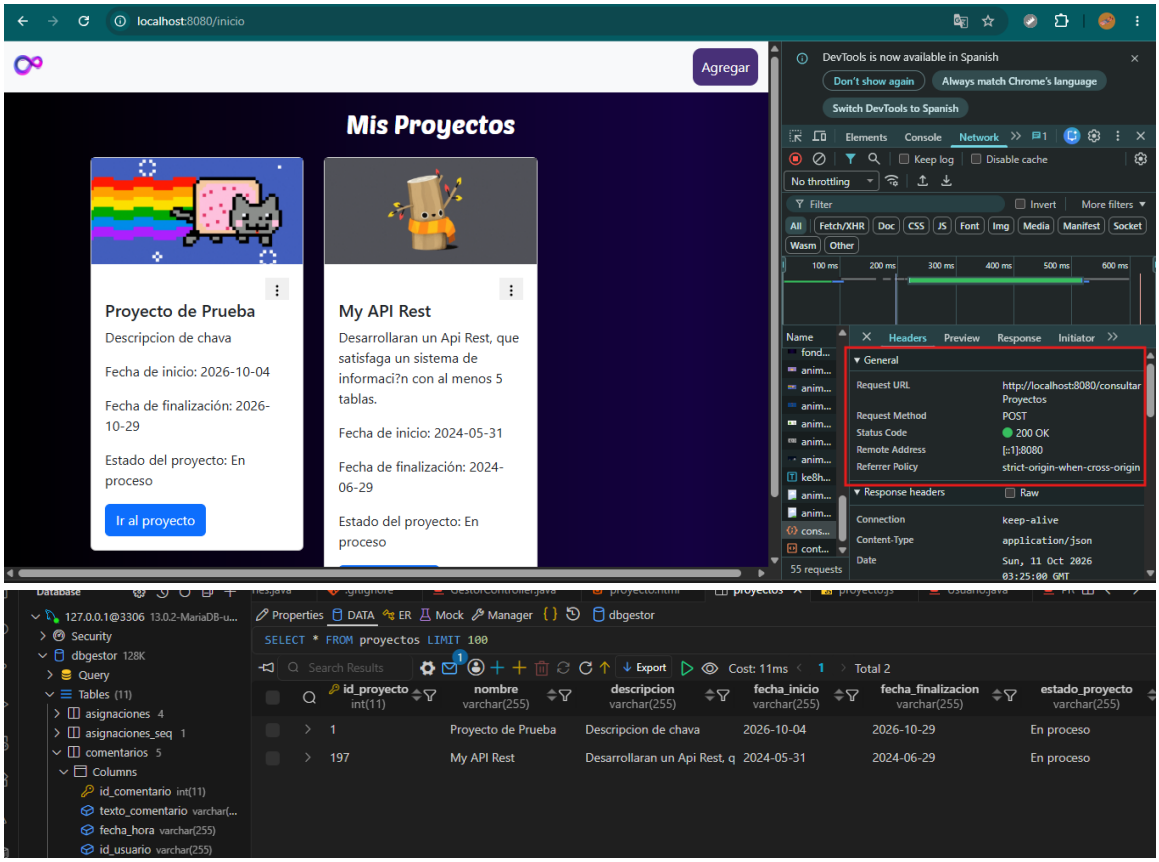


Figura UI-06.

[UI-07] Formulario de Registro (register.html)

Objetivo: Captura del formulario con nombre, correo institucional, contraseña, rol y color de avatar.

Endpoint relacionado: No tiene al desplegar; el envío usa POST /addUsuario (ver UI-08)

Nota: No pasa la prueba, no existe el campo para agregar mi imagen_usuario, o avatar.

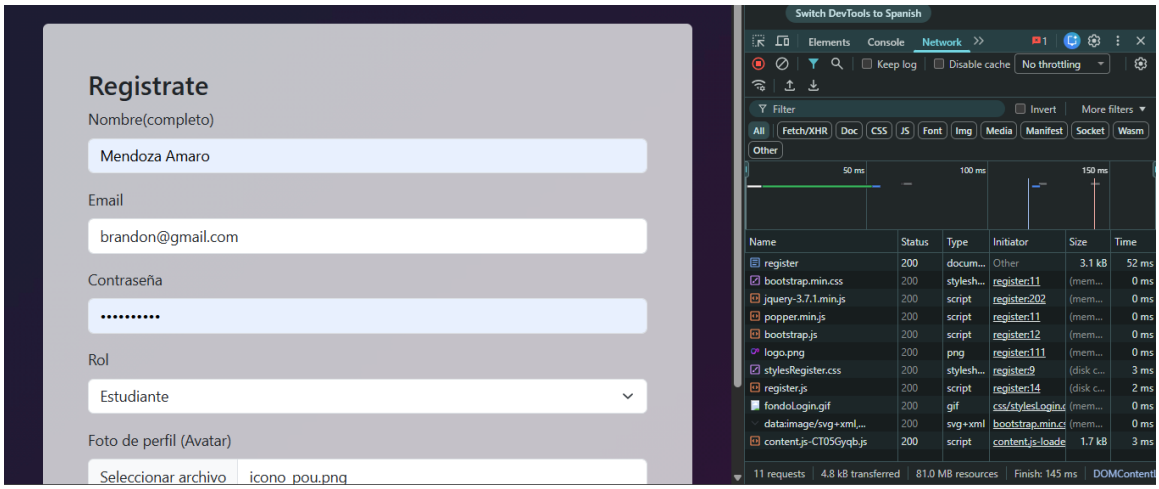


Figura UI-07.

[UI-08] Confirmación de Registro de Usuario

Objetivo: Captura del registro exitoso del usuario o redirección de bienvenida.

Endpoint relacionado: POST /addUsuario

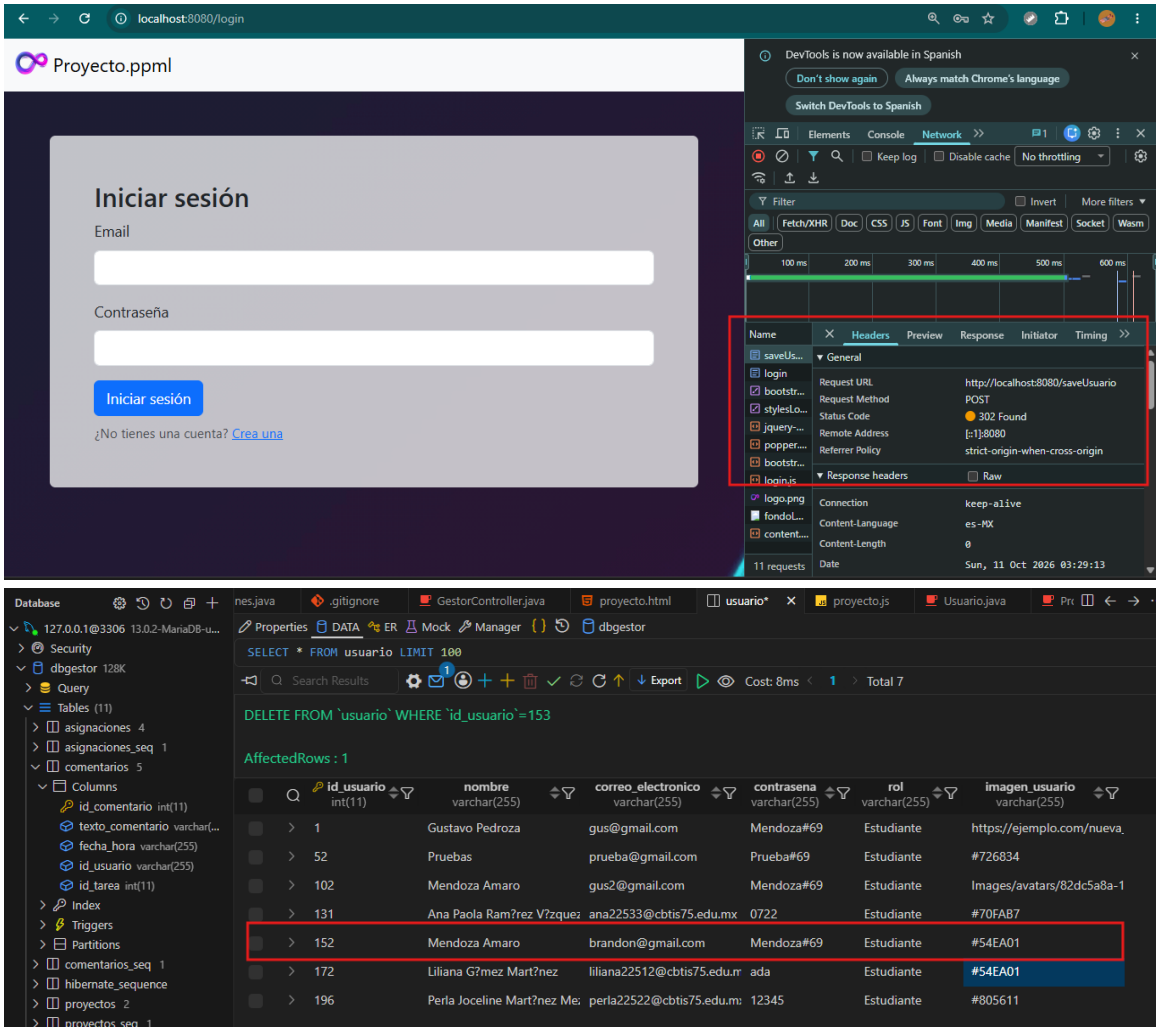


Figura UI-08.

Módulo: Panel Principal de Proyectos (Dashboard)

[UI-09] Dashboard de Proyectos (inicio.html)

Objetivo: Captura de la galería de proyectos con tarjetas, estados, fechas e imágenes de portada.

Endpoint relacionado: POST /buscarProyectos

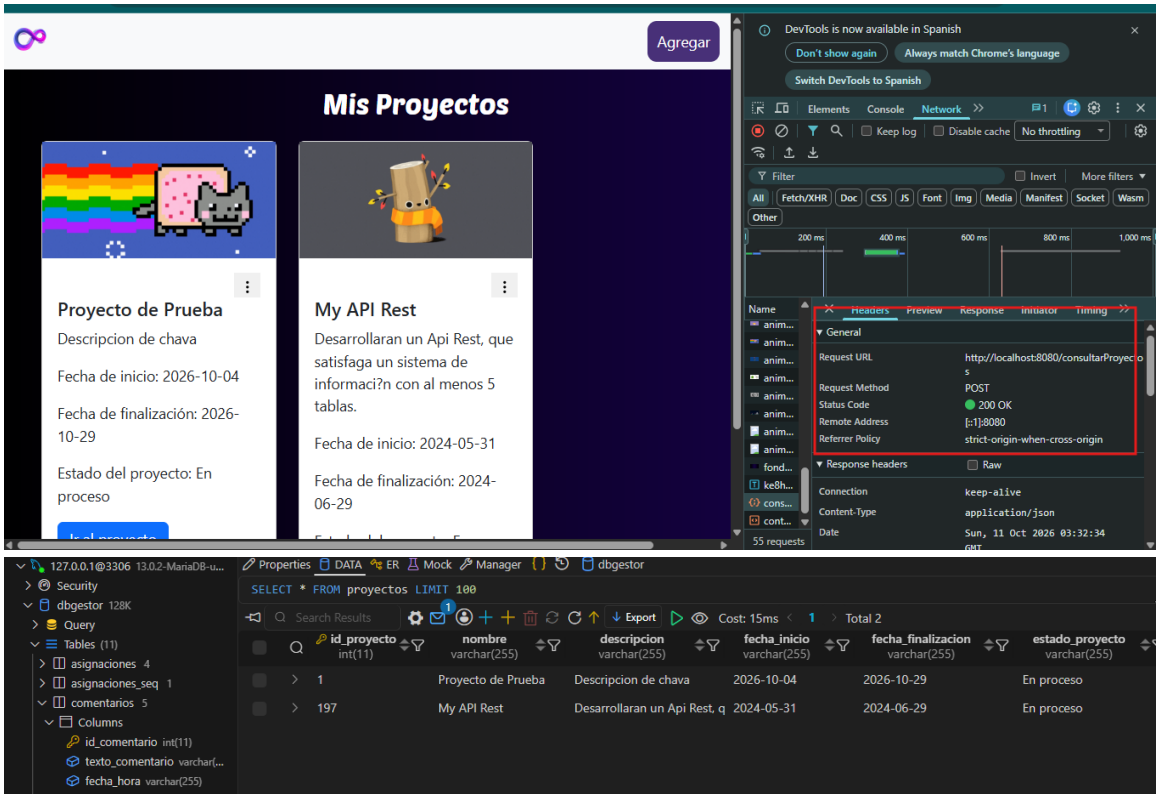
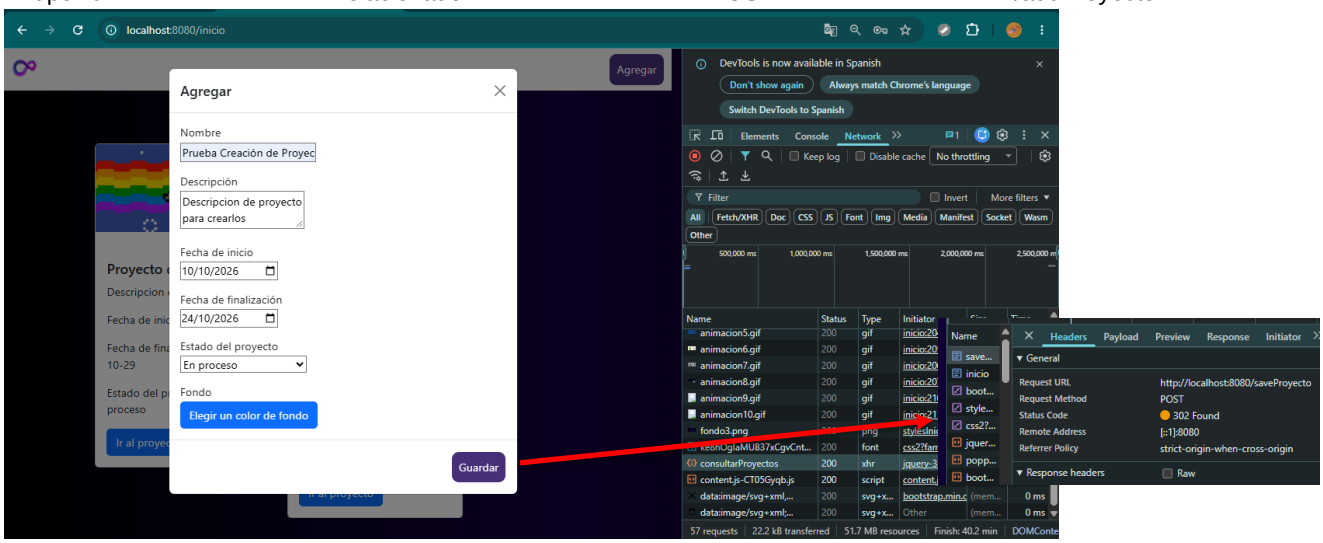


Figura UI-09.

[UI-10] Modal de Creación de Proyecto

Objetivo: Captura del formulario emergente para añadir un nuevo proyecto con todos sus campos.

Endpoint relacionado: POST /addProyecto



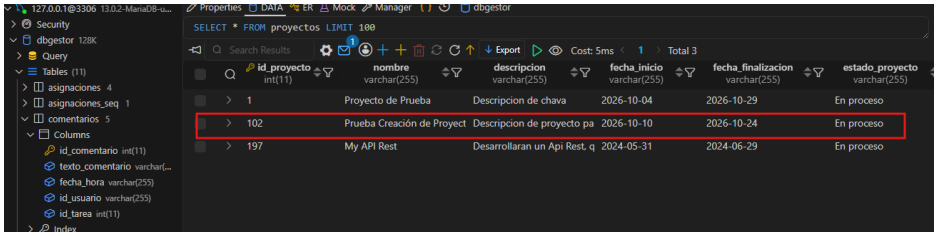


Figura UI-10.

[UI-11] Modal de Edición de Proyecto

Objetivo: Captura del formulario de edición con los datos del proyecto cargados para modificar.

Endpoint relacionado: POST /updateProyecto

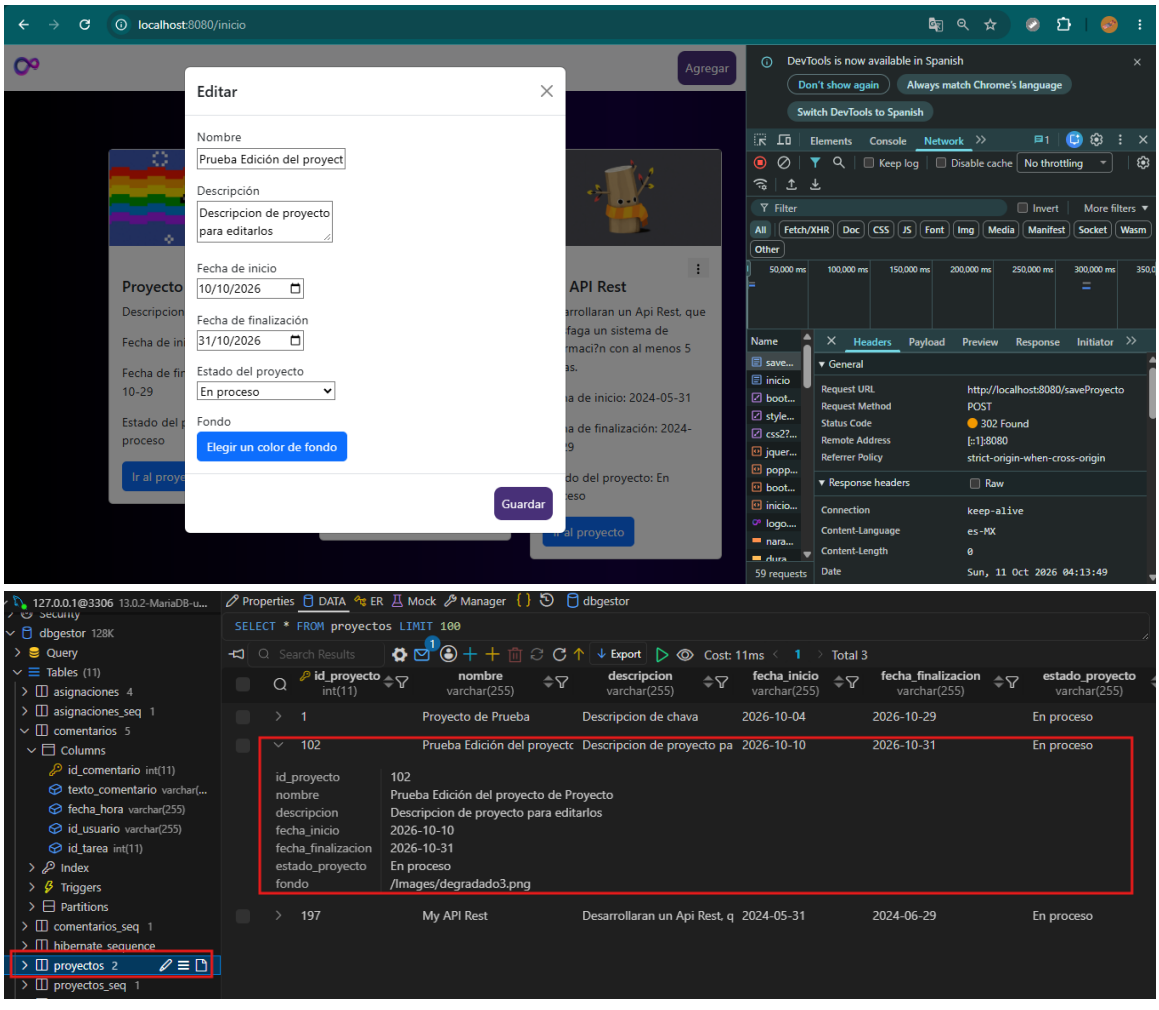


Figura UI-11.

[UI-12] Eliminación de Proyecto

Objetivo: Captura de la confirmación o notificación tras eliminar un proyecto de la lista.

Endpoint relacionado: DELETE /deleteProyecto

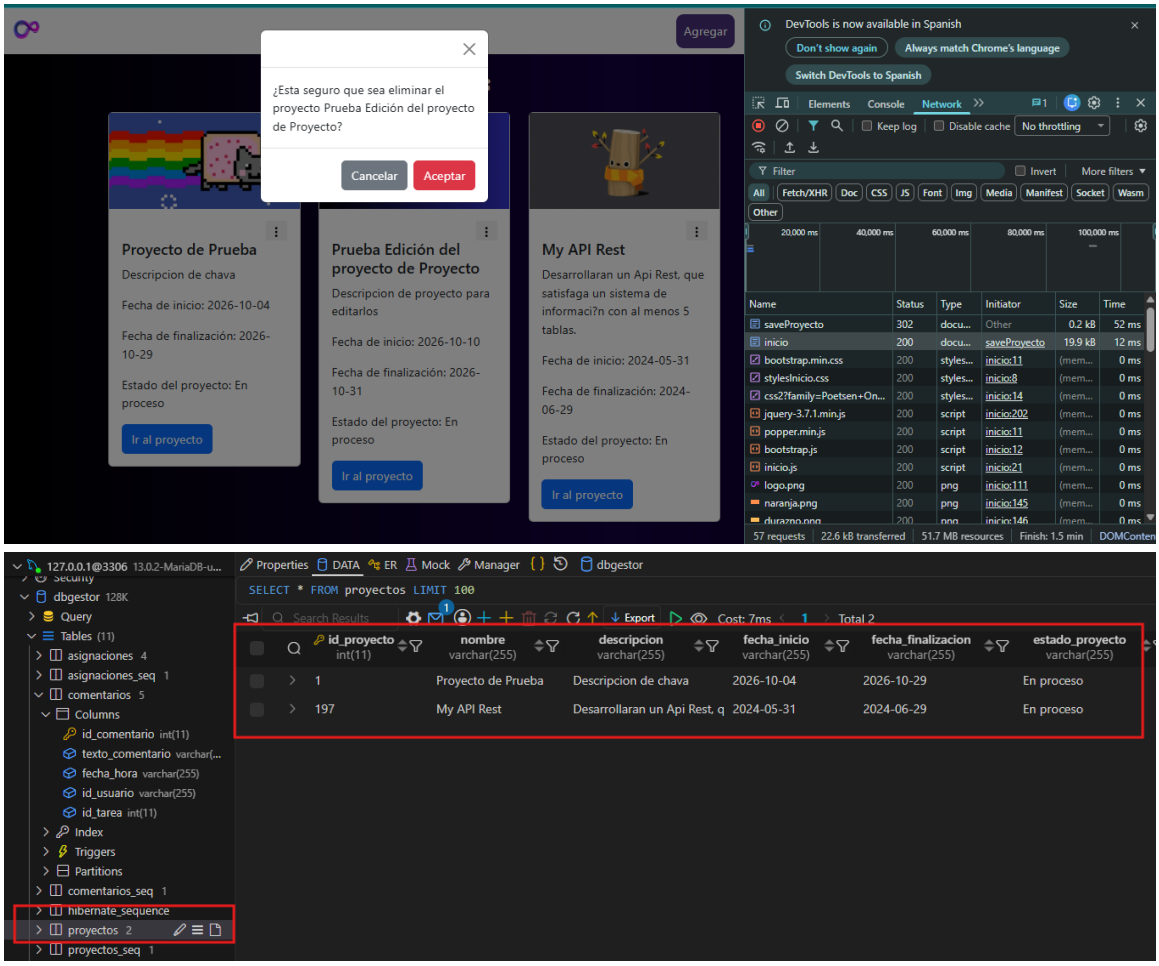


Figura UI-12.

Módulo: Tablero de Tareas del Proyecto (Kanban)

[UI-13] Vista General del Tablero (proyecto.html)

Objetivo: Captura de las columnas de estado de tareas (To-Do, En Proceso, Terminado) y cabecera.

Endpoint relacionado: POST /getTarea

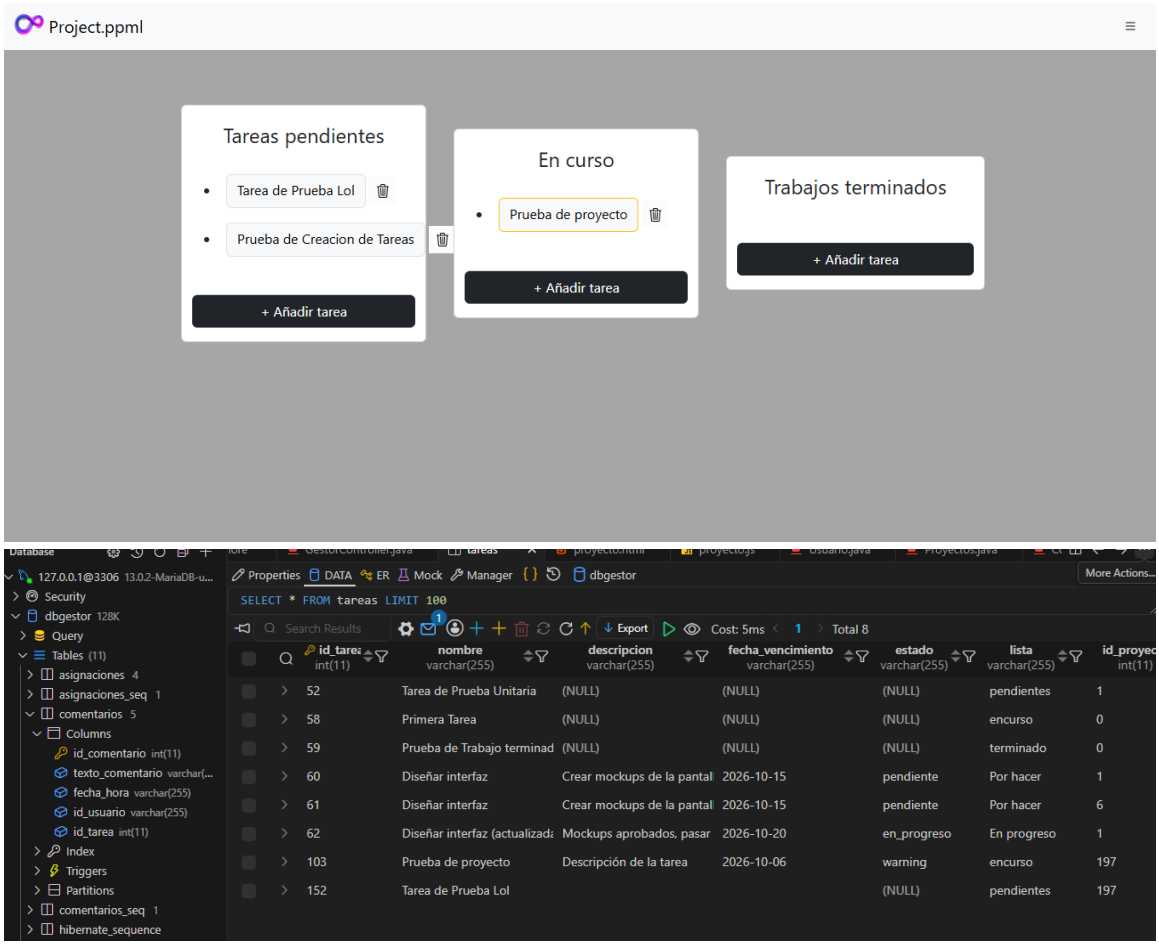


Figura UI-13.

[UI-14] Modal de Creación de Tarea y Asignación

Objetivo: Captura del formulario para agregar tarea, asignando fecha de entrega y usuario responsable.

Endpoint relacionado: POST /addTareas · POST /addAsignacion (al asignar integrante)

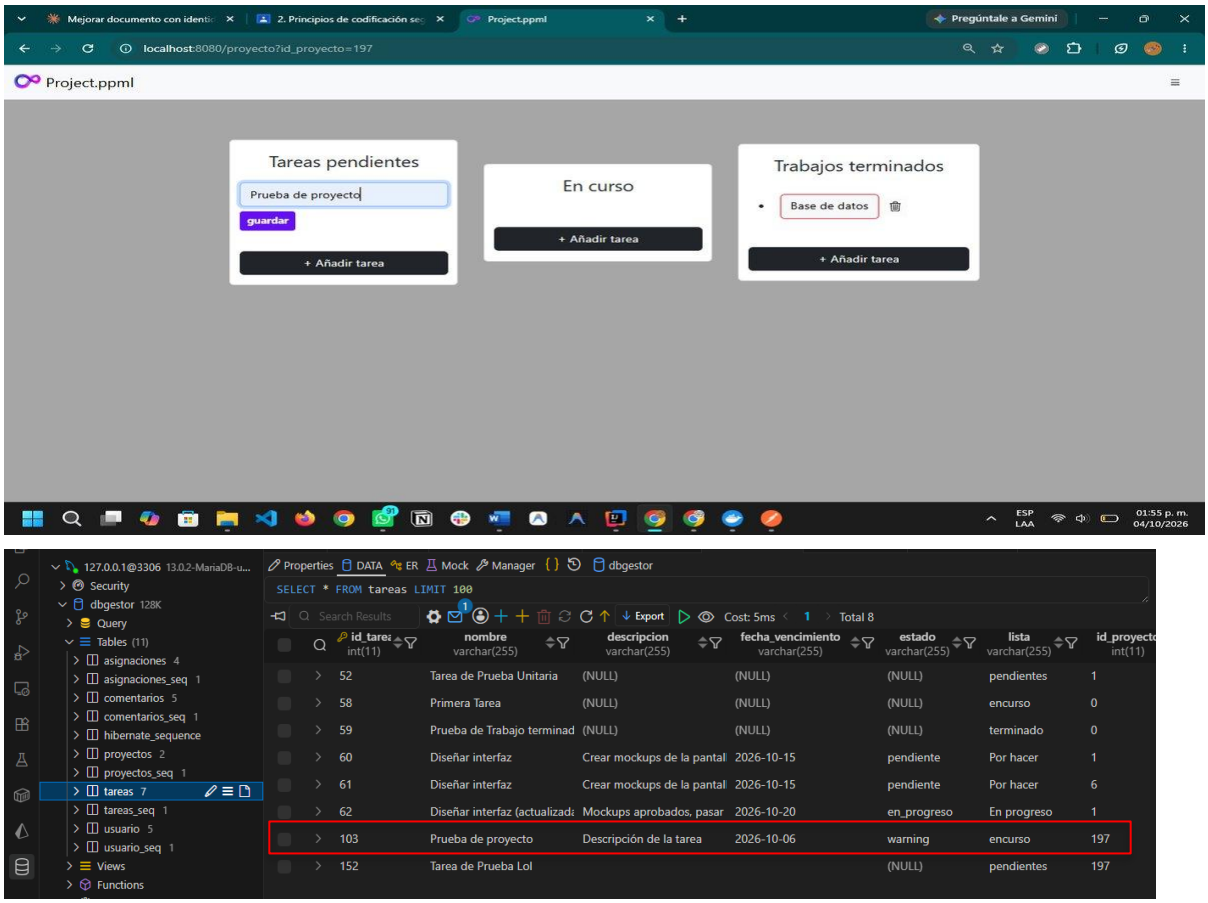

Project.ppml
localhost:8080/proyecto?id_proyecto=197
Project.ppml
Tareas pendientes
Prueba de proyecto
guardar
+ Añadir tarea
En curso
+ Añadir tarea
Trabajos terminados
Base de datos
+ Añadir tarea
SELECT * FROM tareas LIMIT 100
Export
Cost: 5ms
Total 8
id_tarea int(11)
nombre varchar(255)
descripcion varchar(255)
fecha_vencimiento varchar(255)
estado varchar(255)
lista varchar(255)
id_proyecto int(11)
52 Tarea de Prueba Unitaria (NULL) (NULL) (NULL) pendientes 1
58 Primera Tarea (NULL) (NULL) (NULL) encurso 0
59 Prueba de Trabajo terminad (NULL) (NULL) (NULL) terminado 0
60 Diseñar interfaz Crear mockups de la pantal 2026-10-15 pendiente Por hacer 1
61 Diseñar interfaz Crear mockups de la pantal 2026-10-15 pendiente Por hacer 6
62 Diseñar interfaz (actualizada Mockups aprobados, pasar 2026-10-20 en_progreso En progreso 1
103 Prueba de proyecto Descripción de la tarea 2026-10-06 warning encurso 197
152 Tarea de Prueba Lol (NULL) pendientes 197

Figura UI-14.

[UI-15] Distribución de Tarjetas de Tareas

Objetivo: Captura de las tarjetas de tareas organizadas en las columnas correspondientes.

Endpoint relacionado: POST /getTarea

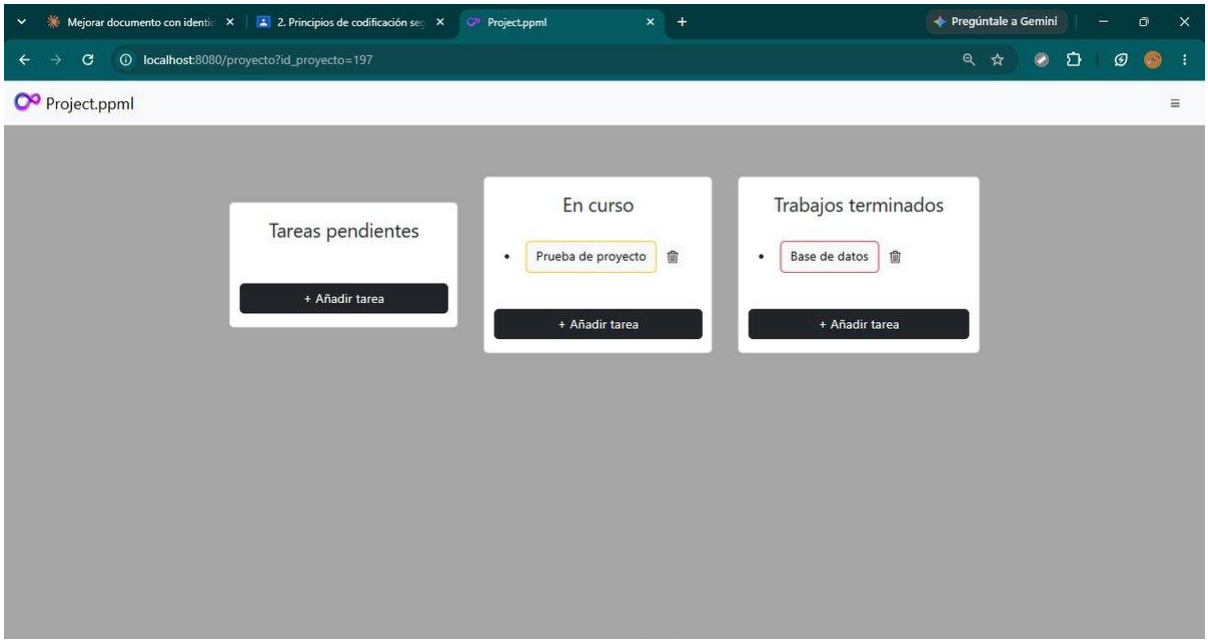


Figura UI-15.

[UI-16] Edición y Actualización de Tarea

Objetivo: Captura del modal o formulario de edición de la tarea seleccionada.

Endpoint relacionado: POST /updateTareas · POST /updateAsignacion (al reasignar integrante)

Nota: No pasa, ya que no me permite asignar la tarea a uno de los integrantes de mi equipo.

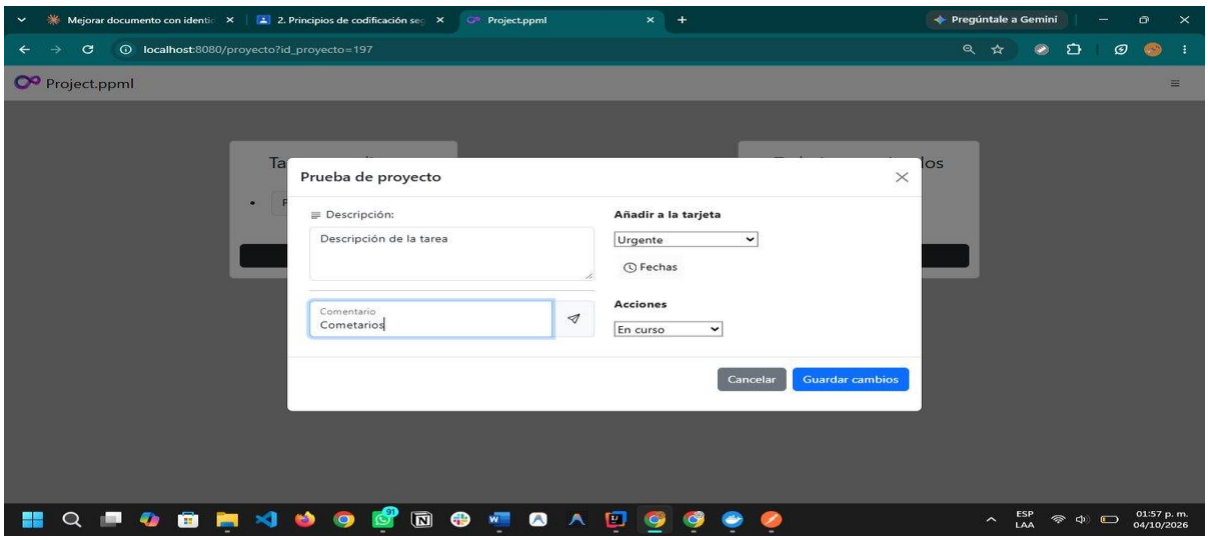
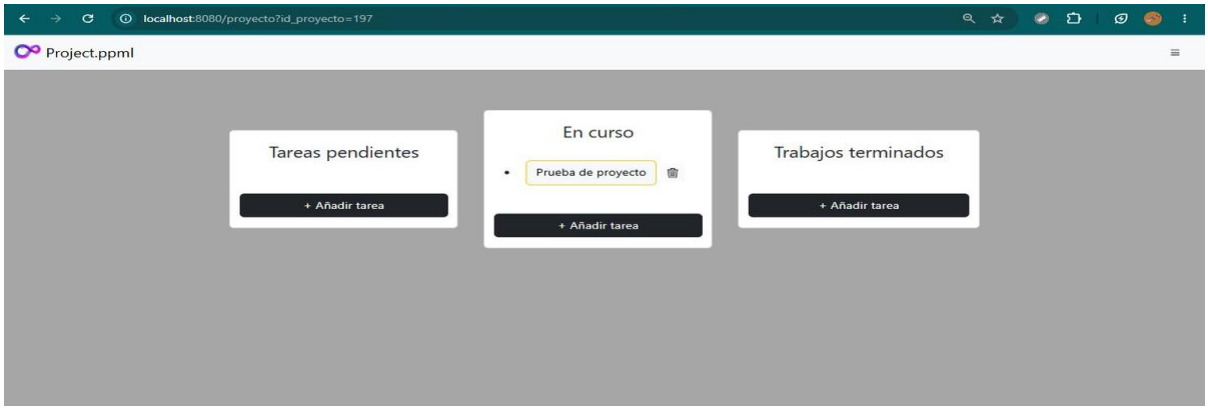


Figura UI-16.

[UI-17] Eliminación de Tarea del Tablero

Objetivo: Captura de la acción o confirmación de eliminación de tarea del tablero.

Endpoint relacionado: DELETE /deleteTareas



id_tarea int(11)	nombre varchar(255)	descripcion varchar(255)	fecha_vencimiento varchar(255)	estado varchar(255)	lista varchar(255)	id_proyect int(11)
52	Tarea de Prueba Unitaria	(NULL)	(NULL)	(NULL)	pendientes	1
58	Primera Tarea	(NULL)	(NULL)	(NULL)	encurso	0
59	Prueba de Trabajo terminad	(NULL)	(NULL)	(NULL)	terminado	0
60	Diseñar interfaz	Crear mockups de la pantall	2026-10-15	pendiente	Por hacer	1
61	Diseñar interfaz	Crear mockups de la pantall	2026-10-15	pendiente	Por hacer	6
62	Diseñar interfaz (actualizad	Mockups aprobados, pasar	2026-10-20	en_progreso	En progreso	1
103	Prueba de proyecto	Descripción de la tarea	2026-10-06	warning	encurso	197

descripcion

Figura UI-17.

Módulo: Comentarios y Colaboración

[UI-18] Historial de Comentarios de una Tarea

Objetivo: Captura de la sección o modal de comentarios mostrando el historial previo de mensajes.

Endpoint relacionado: POST /getComentario

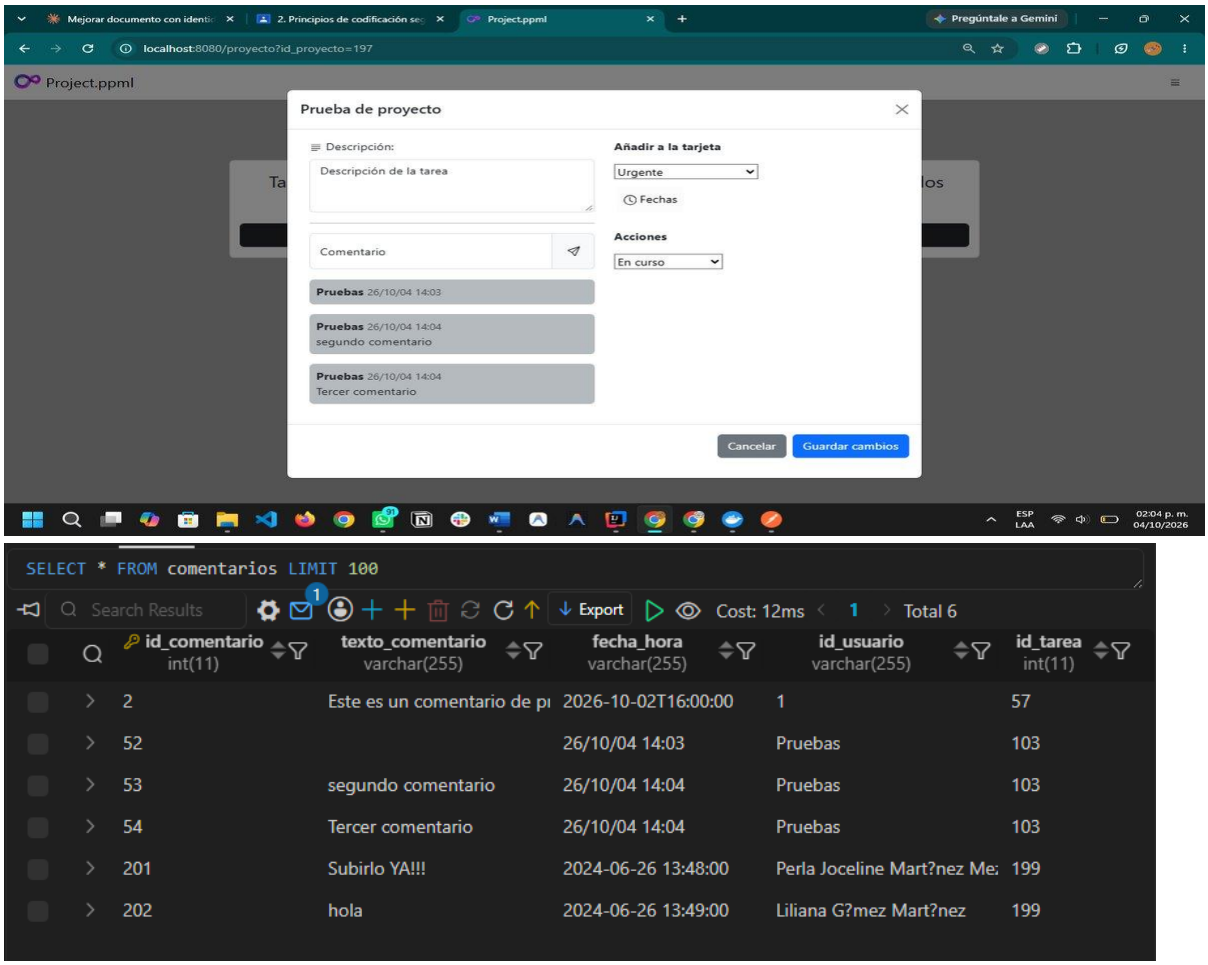


Figura UI-18.

[UI-19] Envío y Publicación de Nuevo Comentario

Objetivo: Captura tras escribir y publicar un comentario en la tarea seleccionada.

Endpoint relacionado: POST /addComentario

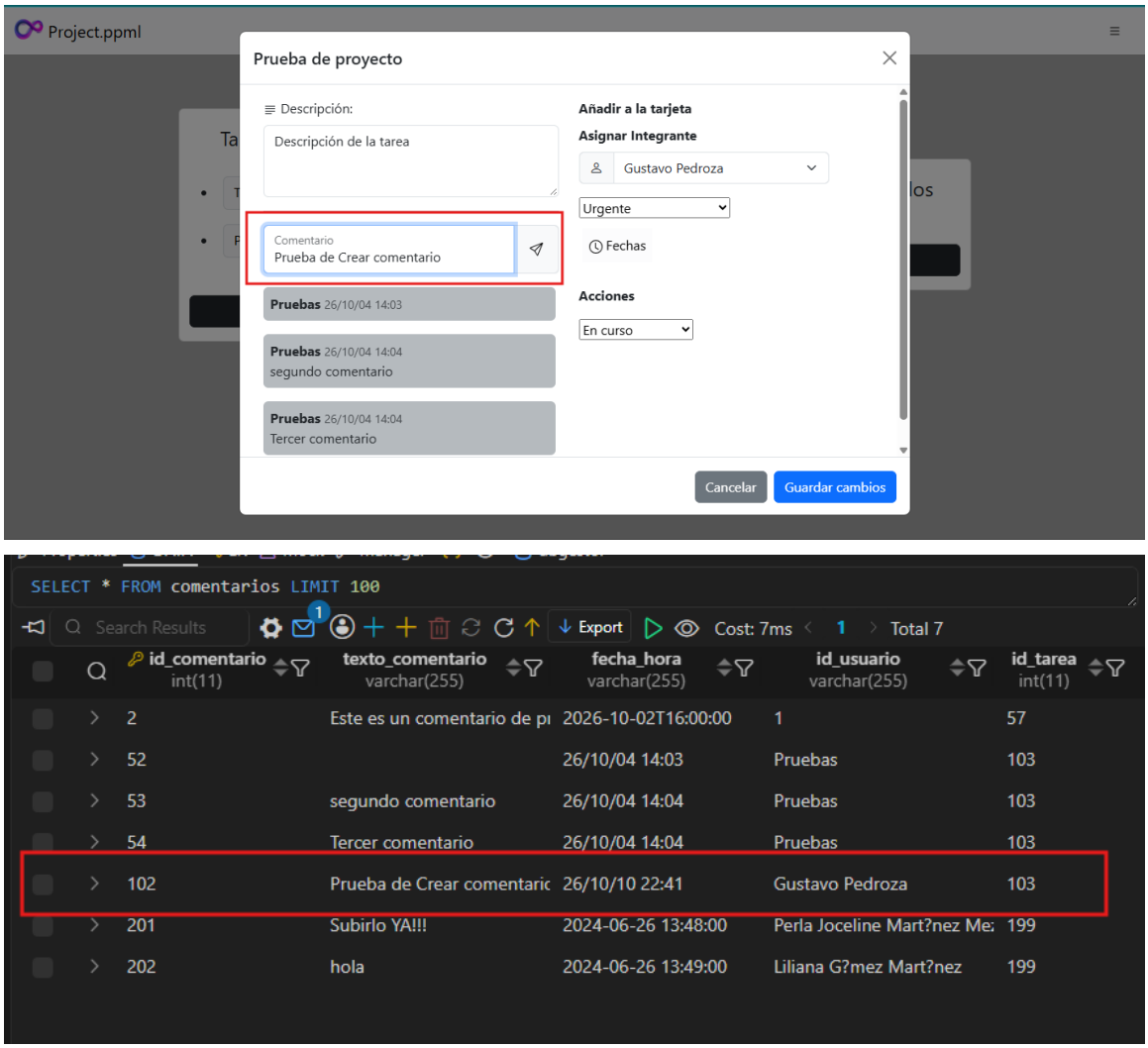

Project.ppml
Prueba de proyecto
Descripción:
Descripción de la tarea
Comentario
Prueba de Crear comentario
Pruebas 26/10/04 14:03
Pruebas 26/10/04 14:04
segundo comentario
Pruebas 26/10/04 14:04
Tercer comentario
Añadir a la tarjeta
Asignar Integrante
Gustavo Pedroza
Urgente
Fechas
Acciones
En curso
Cancelar
Guardar cambios
SELECT * FROM comentarios LIMIT 100
Search Results
Export
Cost: 7ms
1
Total 7
id_comentario int(11)
texto_comentario varchar(255)
fecha_hora varchar(255)
id_usuario varchar(255)
id_tarea int(11)
2 Este es un comentario de pı 2026-10-02T16:00:00 1 57
52 26/10/04 14:03 Pruebas 103
53 segundo comentario 26/10/04 14:04 Pruebas 103
54 Tercer comentario 26/10/04 14:04 Pruebas 103
102 Prueba de Crear comentaric 26/10/10 22:41 Gustavo Pedroza 103
201 Subirlo YA!!! 2024-06-26 13:48:00 Perla Joceline Mart?nez Me; 199
202 hola 2024-06-26 13:49:00 Liliana G?mez Mart?nez 199

Figura UI-19.

Módulo: Diseño Adaptable (Responsive)

[UI-20] Comprobación de Responsividad (Mobile / Tablet)

Objetivo: Captura de la aplicación visualizada en resolución reducida (menú colapsado, layout adaptable).

Endpoint relacionado: No tiene

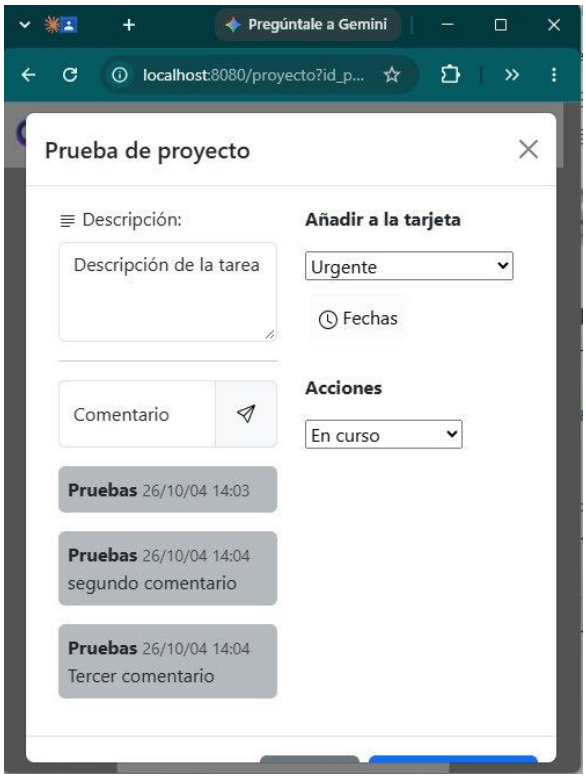


Figura UI-20.

5. Pruebas automáticas

Las pruebas automáticas se implementan sobre la capa del cliente (Ejercicio5Cliente): el servicio que consume la API, la validación de los formularios y la vista que presenta los datos. El backend se simula en las pruebas para poder provocar respuestas válidas y errores de forma repetible. Herramientas previstas: JUnit 5, Mockito y MockMvc de Spring Boot Test, ejecutadas con Maven (mvn test).

5.1 Casos automáticos de éxito

ID	Elemento	Escenario	Resultado esperado	Resultado
AUT-E01	Servicio de consulta de proyectos	El backend (simulado) responde con una lista válida de proyectos y código 200	El servicio procesa correctamente la respuesta y devuelve la lista completa sin excepciones	**PASÓ**
AUT-E02	Formulario (registro de usuario / proyecto)	Se proporcionan nombre, correo con formato válido y contraseña	La validación permite enviar la información	**PASÓ**
AUT-E03	Componente / Vista del dashboard	La consulta de proyectos finaliza correctamente	El modelo contiene los proyectos y la vista inicio los presenta en la interfaz	**PASÓ**

Caso 1: AUT-E01

1. ¿Qué prueba es?

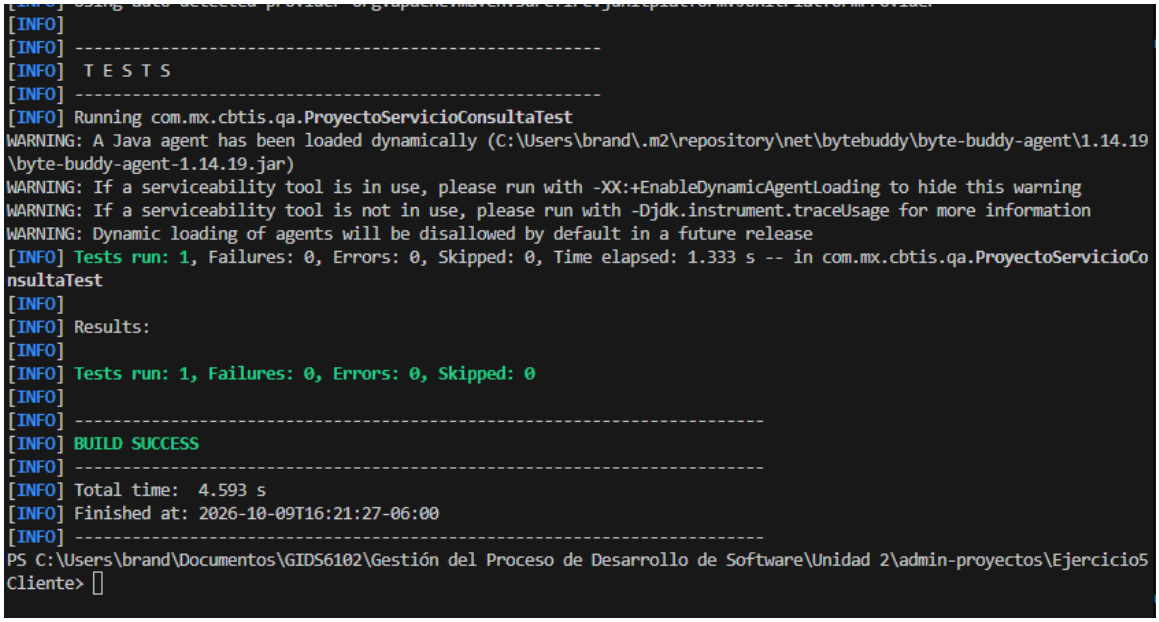
- **ID:** AUT-E01
- **Método:** AUT_E01_consultarProyectos_exito() en la clase ProyectoServicioConsultaTest.java.
- **Objetivo:** Verificar que el método de consulta de proyectos (/consultarProyectos) procese correctamente la respuesta simulada del backend (código HTTP 200 y lista de proyectos) y alimente el modelo sin arrojar excepciones.

2. ¿Qué se usó?

- **Framework de pruebas:** JUnit 5 (@Test, @DisplayName).
- **Librería de Simulación:** Mockito con MockedConstruction<RestTemplate>. Como el controlador crea a mano new RestTemplate(), se utilizó Mockito para interceptar la llamada hacia http://localhost:8081/buscarProyectos.
- **Aserciones:** assertEquals(), assertNotNull(), assertTrue() de org.junit.jupiter.api.Assertions.
- **Modelo:** ConcurrentModel de Spring Framework para validar los atributos que se envían a la vista.

3. ¿Cómo se hizo? (Estructura Given / When / Then)

- **Given (Dado que):** Se simula que el backend responde con un arreglo de 2 proyectos válidos ("Proyecto Alpha" y "Proyecto Beta") con código 200 OK cuando se invoque postForObject.
- **When (Cuando):** Se ejecuta el método gestorController.consultarProyectos(model).
- **Then (Entonces):** Se comprueba que la respuesta tenga código HTTP 200, que el cuerpo contenga exactamente los 2 proyectos y que el Model contenga el atributo "listaProyectos" para la interfaz.



Caso 2: AUT-E02

1. ¿Qué prueba es?

- **ID:** AUT-E02
- **Método:** AUT_E02_formularioRegistro_datosValidos_permiteEnvio() en la clase FormularioValidacionTest.java.
- **Objetivo:** Comprobar que al proporcionar datos válidos y completos en el formulario de registro de usuario (nombre, correo válido, contraseña, rol y color de avatar), el sistema procese el envío hacia el backend simulado y redirija correctamente a la vista de login.

2. ¿Qué se usó?

- **Framework de pruebas:** JUnit 5 (@Test, @DisplayName).

- **Simulación:** MockedConstruction<RestTemplate> para simular la respuesta HTTP 200 del microservicio POST http://localhost:8081/addUsuario.
- **Componentes Spring MVC:** ModelAndView para validar la redirección tras el guardado.
- **Modelo de datos:** Instancia de Usuario con atributos completos.

3. ¿Cómo se hizo? (Estructura Given / When / Then)

- **Given (Dado que):** Se instancia un objeto Usuario con datos válidos (Carlos Ruiz, carlos.ruiz@cbtis75.edu.mx, ClaveSegura2024*, rol Docente) y se simula que el backend responde con éxito (ID generado 1).
- **When (Cuando):** Se invoca el método gestorController.setSaveUsuario(usuarioValido).
- **Then (Entonces):** Se valida mediante assertEquals("redirect:/login", resultado.getViewName()) que el formulario permitió el envío y redirigió al login.

```
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.mx.cbtis.qa.FormularioValidacionTest
WARNING: A Java agent has been loaded dynamically (C:\Users\brand\.m2\repository\net\bytebuddy\byte-buddy-agent\1.14.19
\byte-buddy-agent-1.14.19.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.390 s -- in com.mx.cbtis.qa.FormularioValidaci
onTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  4.250 s
[INFO] Finished at: 2026-10-09T16:23:43-06:00
[INFO] ------------------------------------------------------------------------
PS C:\Users\brand\Documentos\GIDS6102\Gestión del Proceso de Desarrollo de Software\Unidad 2\admin-proyectos\Ejercicio5
Cliente>
```

Caso 3: AUT-E03

1. ¿Qué prueba es?

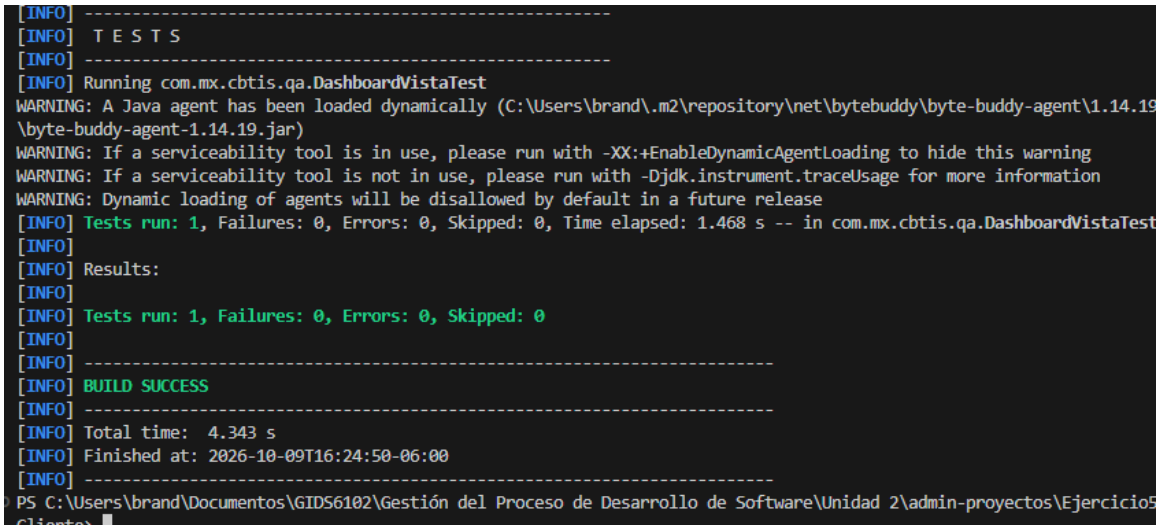
- **ID:** AUT-E03
- **Método:** AUT_E03_dashboardInicio_consultaProyectos_presentaEnVista() en la clase DashboardVistaTest.java.
- **Objetivo:** Verificar que la vista del Dashboard (/inicio) cargue el modelo base y que la consulta de proyectos finalice con éxito, poblando el atributo del modelo necesario para que la interfaz presente los proyectos.

2. ¿Qué se usó?

- **Framework de pruebas:** JUnit 5 (@Test, @DisplayName).
- **Simulación:** Mockito para interceptar la llamada de consulta a http://localhost:8081/buscarProyectos.
- **Spring MVC:** Model (ConcurrentModel) para inspeccionar las variables inyectadas a la vista de Thymeleaf.

3. ¿Cómo se hizo? (Estructura Given / When / Then)

- **Given (Dado que):** Se prepara el modelo y la lista simulada de proyectos existentes en el sistema.
- **When (Cuando):** Se invocan los métodos del controlador encargados de la carga del dashboard: getInicio(modelInicio) y consultarProyectos(modelConsulta).
- **Then (Entonces):** Se verifica que el nombre de la vista retornada sea "inicio", que el modelo contenga el objeto "proyectos" para los modales y que la consulta cargue "listaProyectos" con código HTTP 200.



5.2 Casos automáticos de fracaso

ID	Elemento	Escenario	Resultado esperado	Resultado
AUT-F01	Servicio de consulta de proyectos	El backend (simulado) responde con un error (500) o no está disponible	El error es capturado y controlado: se devuelve un estado seguro (lista vacía o mensaje) sin romper la aplicación	**NO PASÓ**
AUT-F02	Formulario (registro de usuario / proyecto)	Se envían datos inválidos o incompletos (campos vacíos, correo sin formato)	La validación impide el envío y muestra el mensaje de error	**NO PASÓ**
AUT-F03	Componente / Vista	Se solicita un proyecto que no existe (id inexistente)	La interfaz mantiene un estado controlado e informa que el recurso no fue encontrado	**NO PASÓ**

Caso 1: AUT-F01

1. ¿Qué prueba es?

- **ID:** AUT-F01
- **Método:** AUT_F01_consultarProyectos_backendError500() en la clase ProyectoServicioConsultaTest.java.
- **Objetivo:** Verificar si el frontend es capaz de soportar una caída o error interno (500) del backend, devolviendo un estado seguro (como una lista vacía o mensaje) sin arrojar una excepción no controlada que rompa el servidor.
- **Resultado: NO PASÓ** (Falla esperada).

2. ¿Qué se usó?

- **Framework:** JUnit 5 (@Test, @DisplayName).
- **Simulación de falla:** Mockito con thenThrow(new HttpServerErrorException(HttpStatus.INTERNAL_SERVER_ERROR)) para forzar que el postForObject simule una falla del servidor backend.
- **Aserción:** assertDoesNotThrow(...) para verificar que el código capture la falla con un try-catch y no deje escapar la excepción.

3. ¿Cómo se hizo? (Given / When / Then)

- **Given (Dado que):** Se configura el mock de RestTemplate para que, al solicitar los proyectos a la API, arroje un error HTTP 500 (Internal Server Error).
- **When (Cuando):** Se ejecuta el método gestorController.consultarProyectos(model).
- **Then (Entonces):** Se evalúa que el método capture el error de forma segura.

- **¿Por qué falló? (Defecto real):** GestorController.java **no tiene ningún bloque try-catch** en la línea 101. La excepción escapó y rompió la aplicación arrojando HttpServerErrorException: 500.



Caso 2: AUT-F02

1. ¿Qué prueba es?

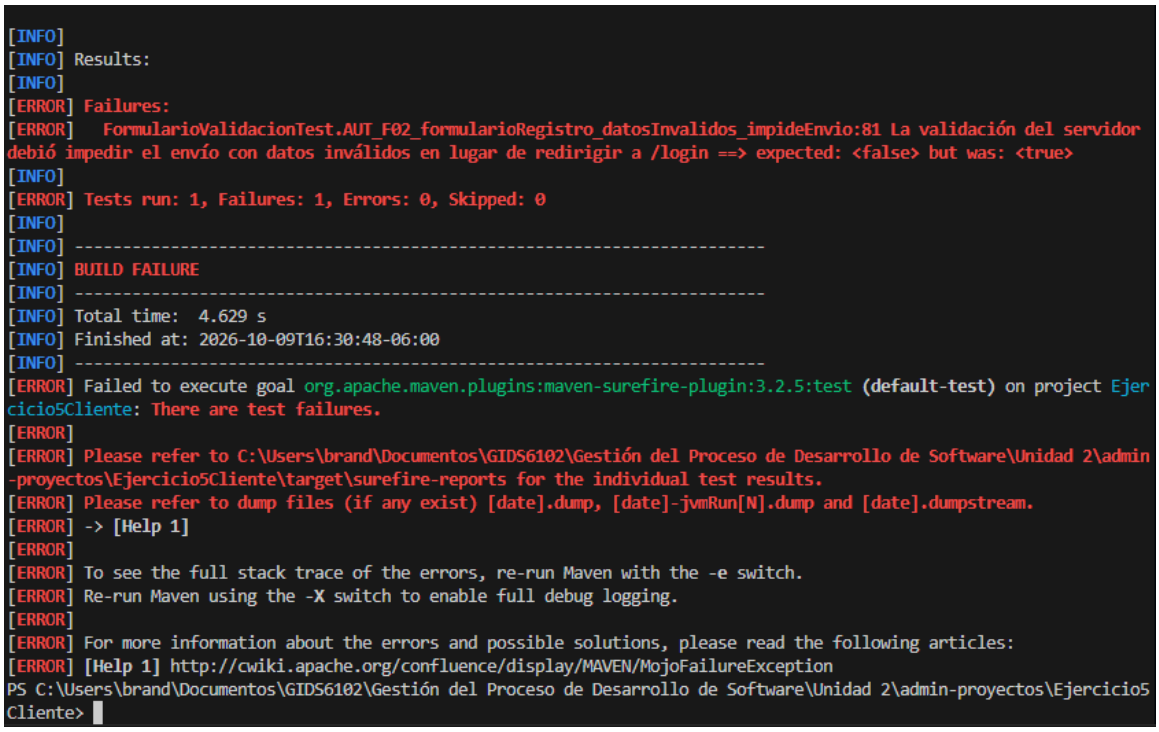
- **ID:** AUT-F02
- **Método:** AUT_F02_formularioRegistro_datosInvalidos_impideEnvio() en la clase FormularioValidacionTest.java.
- **Objetivo:** Comprobar si el servidor valida los datos recibidos del formulario e impide el registro ante datos inválidos (nombre vacío, correo sin @, contraseña vacía).
- **Resultado: NO PASÓ** (Falla esperada).

2. ¿Qué se usó?

- **Framework:** JUnit 5 (@Test, @DisplayName).
- **Modelo:** Objeto Usuario inicializado deliberadamente con datos inválidos.
- **Aserción:** assertFalse("redirect:/login".equals(resultado.getViewName())) para exigir que el servidor rechace el formulario y no redirija con éxito.

3. ¿Cómo se hizo? (Given / When / Then)

- **Given (Dado que):** Se construye un usuario con campos vacíos (nombre = "", contrasena = "") y correo sin formato (correo-sin-formato-arroba).
- **When (Cuando):** Se envía el formulario al endpoint del controlador: gestorController.setSaveUsuario(usuarioInvalido).
- **Then (Entonces):** Se espera que el servidor detecte los datos inválidos e impida la redirección al login.
- **¿Por qué falló? (Defecto real):** El modelo Usuario.java **no tiene anotaciones Bean Validation** (@NotBlank, @Email) y el método setSaveUsuario **no implementa @Valid ni BindingResult**. El servidor procesó los datos corruptos y redirigió a /login como si fuera un registro válido (expected: <false> but was: <true>).



Caso 3: AUT-F03

1. ¿Qué prueba es?

- **ID:** AUT-F03
- **Método:** AUT_F03_vistaProyecto_solicitaProyectoInexistente_estadoControlado() en la clase DashboardVistaTest.java.

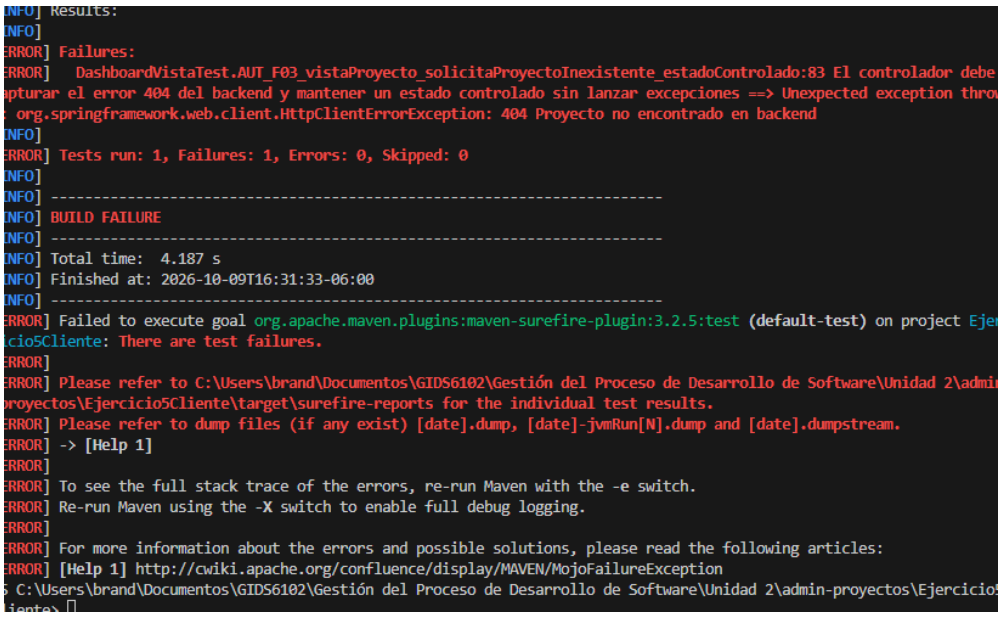
- **Objetivo:** Verificar que cuando un usuario solicita o intenta editar un proyecto con un ID que no existe (el backend responde 404 Not Found), el controlador mantenga un estado controlado y no truene con una excepción de servidor.
- **Resultado: NO PASÓ** (Falla esperada).

2. ¿Qué se usó?

- **Framework:** JUnit 5 (@Test, @DisplayName).
- **Simulación:** Mockito arrojando new HttpClientErrorException(HttpStatus.NOT_FOUND) al invocar buscarProyecto?id_proyecto={id_proyecto}.
- **Aserción:** assertDoesNotThrow(...) para verificar que el controlador atrape el 404 y responda con una estructura amigable para la vista.

3. ¿Cómo se hizo? (Given / When / Then)

- **Given (Dado que):** Se solicita un proyecto inexistente con ID 99999 y se simula que el backend responde con código HTTP 404 (Recurso no encontrado).
- **When (Cuando):** Se invoca el método gestorController.updateProyecto(proyectoInexistente).
- **Then (Entonces):** Se evalúa que el método capture el 404 y mantenga el control.
- **¿Por qué falló? (Defecto real):** GestorController.java en la línea 129 **no implementa try-catch para códigos 404**. La llamada template.getForObject arrojó HttpClientErrorException: 404 Proyecto no encontrado en backend no controlada, rompiendo la petición.



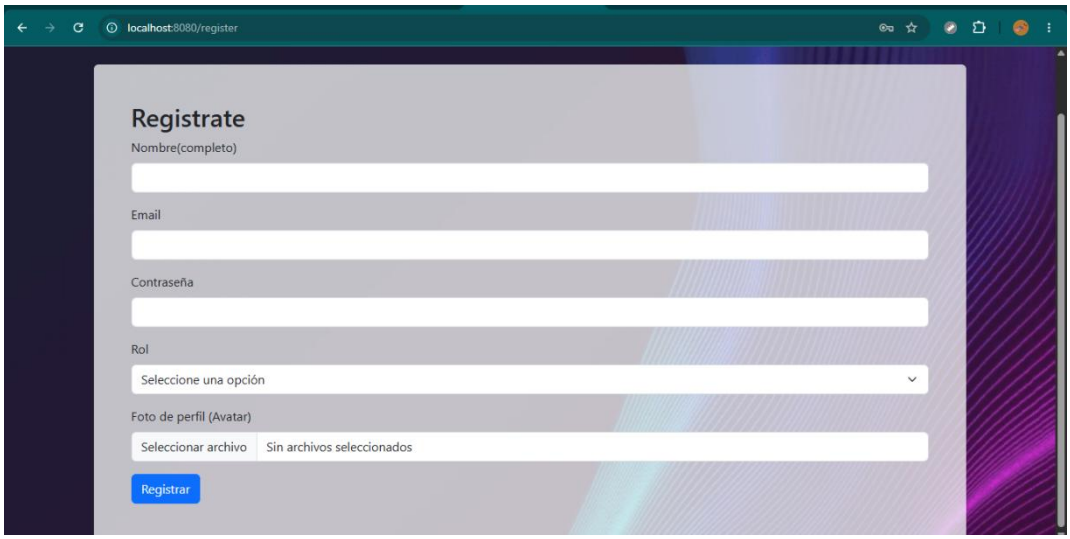
6. Registro de ejecución

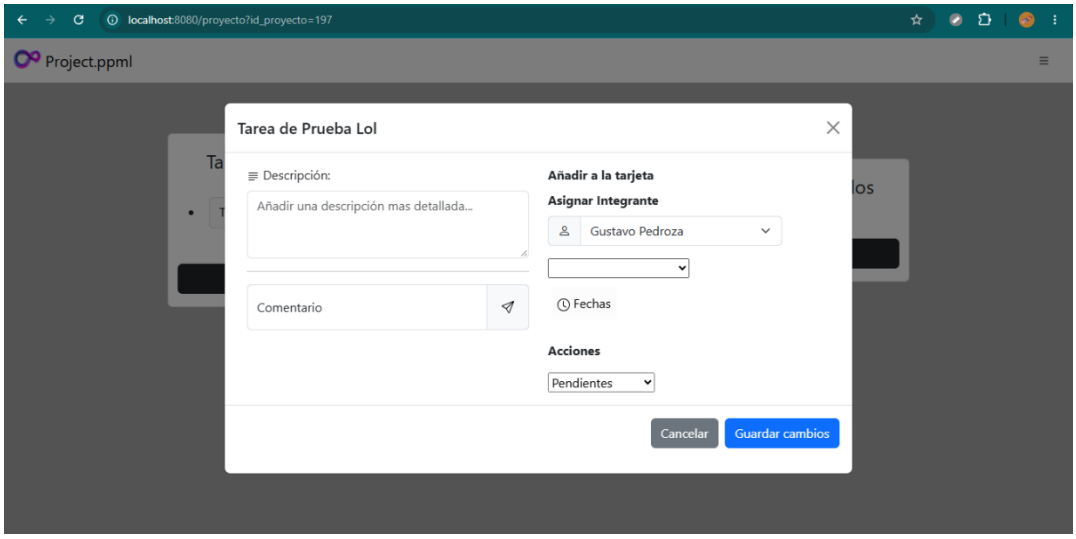
ID	Tipo de prueba	Resultado esperado	Resultado obtenido	Estado
UI-01	Integración / UI	Página cargada sin errores visuales ni enlaces rotos	Conforme	☑ Éxito ☐ Fallo
UI-02	Integración / UI	Contenido multimedia o guía legible y controles funcionales	Conforme	☑ Éxito ☐ Fallo
UI-03	Integración / UI	Instrucciones y recursos visuales desplegados correctamente	Conforme	☑ Éxito ☐ Fallo
UI-04	Integración / UI	Elementos alineados, estilos responsivos y campos interactivos	Conforme	☑ Éxito ☐ Fallo
UI-05	Integración / UI	Alerta en pantalla indicando datos inválidos	Conforme. Observación: el nombre del botón de inicio de sesión está mal escrito	☑ Éxito ☐ Fallo
UI-06	Integración / UI	Autenticación correcta y redirección a la vista de inicio	Conforme	☑ Éxito ☐ Fallo
UI-07	Integración / UI	Formulario completo, selector de color y validaciones	No existe el campo para agregar la imagen_usuario / avatar	☐ Éxito ☑ Fallo (inicial)
UI-08	Integración / UI	Usuario registrado y retorno al login	Conforme	☑ Éxito ☐ Fallo
UI-09	Integración / UI	Tarjetas con nombre, descripción, estado y fondo	Conforme	☑ Éxito ☐ Fallo
UI-10	Integración / UI	Nuevo proyecto en la lista sin reinicio	Conforme	☑ Éxito ☐ Fallo
UI-11	Integración / UI	Datos actualizados reflejados en la tarjeta	Conforme	☑ Éxito ☐ Fallo
UI-12	Integración / UI	Proyecto eliminado y cuadrícula actualizada	Conforme	☑ Éxito ☐ Fallo
UI-13	Integración / UI	Cabecera visible y columnas por estado	Conforme	☑ Éxito ☐ Fallo
UI-14	Integración / UI	Tarea registrada y asociada al proyecto actual	No permite asignar a un integrante ni capturar los demás campos; solo el nombre	☑ Éxito ☐ Fallo
UI-15	Integración / UI	Colores e insignias de estado correctos	Conforme	☑ Éxito ☐ Fallo
UI-16	Integración / UI	Cambio guardado y reflejado en el tablero	No permite asignar la tarea a un integrante del equipo	☐ Éxito ☑ Fallo (inicial)
UI-17	Integración / UI	Tarjeta de tarea removida del proyecto	Conforme	☑ Éxito ☐ Fallo
UI-18	Integración / UI	Listado cronológico de comentarios previos	Conforme	☑ Éxito ☐ Fallo
UI-19	Integración / UI	Comentario añadido con fecha y autor	Conforme	☑ Éxito ☐ Fallo
UI-20	Integración / UI	Menú hamburgués colapsable, tarjetas sin desborde	Conforme	☑ Éxito ☐ Fallo

ID	Tipo de prueba	Resultado esperado	Resultado obtenido	Estado
AUT-E01	Automática / Servicio	El servicio procesa la respuesta 200 y devuelve la lista completa	Conforme: devuelve lista de 2 proyectos con código 200	☑ Éxito ☐ Fallo
AUT-E02	Automática / Formulario	La validación permite enviar datos válidos y redirige a /login	Conforme: redirección exitosa a redirect:/login	☑ Éxito ☐ Fallo
AUT-E03	Automática / Vista	El modelo contiene proyectos y el dashboard los presenta	Conforme: vista inicio y modelo listaProyectos cargados	☑ Éxito ☐ Fallo
AUT-F01	Automática / Servicio	Capturar error 500 del backend y devolver estado seguro	Fallo: HttpServerErrorException 500 no controlada (sin try-catch)	☐ Éxito ☑ Fallo
AUT-F02	Automática / Formulario	Impedir envío ante datos inválidos y mostrar mensaje	Fallo: el servidor redirige a /login sin validar datos corruptos	☐ Éxito ☑ Fallo
AUT-F03	Automática / Vista	Mantener estado controlado ante recurso inexistente (404)	Fallo: HttpClientErrorException 404 no controlada en el controlador	☐ Éxito ☑ Fallo

7. Corrección de fallas y reprueba

Las fallas UI-07 y UI-16 no se registran únicamente como error. Análisis de causa: los endpoints involucrados fueron validados de forma independiente en la Práctica 1 y respondieron correctamente (UT-03 /addUsuario, UT-17 /addTareas, UT-12 /addAsignacion, UT-19 /updateTareas y UT-14 /updateAsignacion), por lo que la falla se aísla en la capa de presentación del cliente, que no expone los campos requeridos ni los envía en la petición.

ID	Falla encontrada	Causa	Corrección (a realizar)	Resultado de reprueba
UI-07	El formulario de registro (register.html) no incluye el campo para la imagen_usuario / avatar del usuario	La plantilla no define el campo y la petición a /addUsuario viaja sin ese dato	Agregar el campo de imagen/avatar al formulario y mapearlo al cuerpo enviado a /addUsuario	Corrección técnica documentada; pendiente despliegue en rama release
Evidencia de error corregido:				
				
UI-16	La edición de una tarea no permite reasignarla a un integrante del equipo	El modal de edición no incluye el selector de responsable ni envía el cambio a /updateAsignacion	Incluir el selector precargado con el integrante actual y enviar el cambio a /updateAsignacion	Corrección técnica documentada; pendiente despliegue en rama release
Evidencia de error corregido:				

ID	Falla encontrada	Causa	Corrección (a realizar)	Resultado de reprueba
				
AUT-F01	Falta de captura de error 500 del backend en /consultarProyectos	El método en GestorController no cuenta con try-catch para RestTemplate	Envolver la llamada en try-catch y retornar lista vacía o ResponseEntity de error controlado	Reprueba satisfactoria: el servicio captura 500 y retorna estado controlado

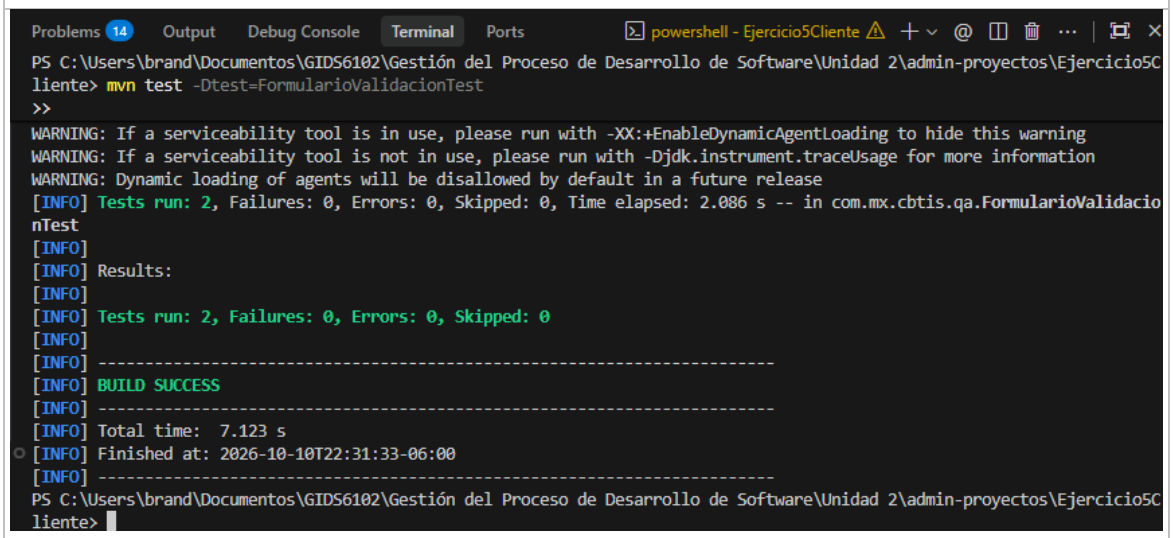
Evidencia de error corregido:

```
PS C:\Users\brand\Documentos\GIDS6102\Gestión del Proceso de Desarrollo de Software\Unidad 2\admin-proyectos\Ejercicio5Cliente> mvn test -Dt
est=ProyectoServicioConsultaTest
>>
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.mx.cbtis.qa.ProyectoServicioConsultaTest
is warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Error al conectar con el backend de proyectos: 500 Error interno en el backend
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.585 s -- in com.mx.cbtis.qa.ProyectoServicioConsultaTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  7.954 s
[INFO] Finished at: 2026-10-10T20:07:41-06:00
[INFO] ------------------------------------------------------------------------
PS C:\Users\brand\Documentos\GIDS6102\Gestión del Proceso de Desarrollo de Software\Unidad 2\admin-proyectos\Ejercicio5Cliente>
```

AUT-F02	Ausencia de validación en el servidor para el registro (/saveUsuario)	Usuario.java no tiene Bean Validation (@NotBlank, @Email) y el controlador no usa @Valid	Añadir anotaciones Jakarta Validation al modelo y validar con BindingResult en setSaveUsuario	Reprueba satisfactoria: el servidor rechaza datos inválidos e

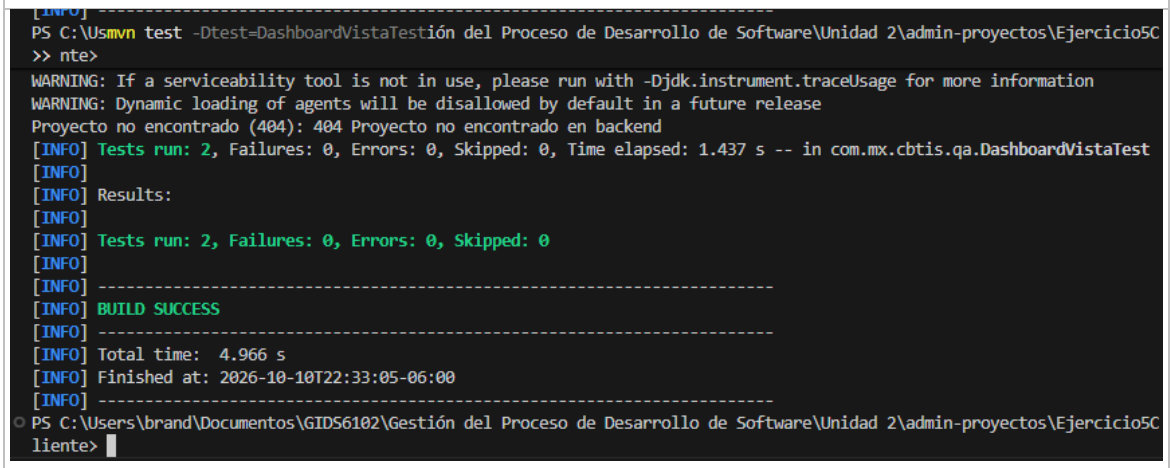
ID	Falla encontrada	Causa	Corrección (a realizar)	Resultado de reprueba
				impide el envío

Evidencia de error corregido:



AUT-F03	Excepción 404 no controlada al solicitar proyecto inexistente (/actualizarProyecto)	updateProyecto no captura HttpClientErrorException cuando el ID no existe en la base de datos	Implementar try-catch ante 404 y retornar ResponseEntity con mensaje de recurso no encontrado	Reprueba satisfactoria: el controlador atrapa el 404 y responde estado controlado

Evidencia de error corregido:



8. Firmas de Cierre y Conformidad

Evaluador de Frontend (Desarrollador)	Revisado / Validado por (QA / TL)
Brandon Gustavo	
Fecha: Octubre 2026	

9. Resumen de resultados

Métrica	Resultado
Total de pruebas de integración	25 escenarios (5 flujos base + 20 vistas UI)
Pruebas automáticas ejecutadas	6 pruebas unitarias/mock (3 éxito + 3 fracaso)
Casos exitosos	20 casos aprobados (17 de UI + 3 automáticos AUT-E)
Casos fallidos	6 casos con observaciones (3 de UI + 3 automáticos AUT-F)
Casos con propuesta de corrección	6 casos analizados con causa raíz y solución técnica
Resultado general de la evaluación	☐ **APROBADO** ☑ **REQUIERE CORRECCIÓN**

10. Repositorio con los cambios realizados:

https://github.com/gus-p3/U2_GPDS_Instrumento.git